

Christian Watts
Ekincan Ufuktepe
ECE-4270 Computer Architecture
03/12/2025

Lab 2 - RISC-V Assembler in C

RISC-V Assembler

This assembler converts assembly code into machine code by reading input files and encoding each instruction according to the RISC-V specification. It's designed to perform a two-pass assembly process. In the first pass, it parses an input file to build a symbol table, distinguishes between the text and data segments, and stores instructions and data in their respective arrays. In the second pass, the simulator translates the parsed instructions into machine code by encoding various instruction types.

Data Structures

- SegType Enum - Used to help program control flow during file processing and determination of the text and data segments.
- Symbol Table - Uses an array of custom *SymEntry* structures to maintain label-address associations. This facilitates resolving branch targets and jump addresses.
- Instruction/Data Arrays - Instructions and data are stored as *InstRec* and *DataRec*, respectively, and stored in their respective arrays.

```
typedef enum { SEG_NONE, SEG_TEXT, SEG_DATA } SegType;
```

```
typedef struct {
    char lbl[MAX_LBL];
    int addr;
} SymEntry;
```

```
typedef struct {
    char instLine[MAX_LINE];
    int addr;
} InstRec;
```

```
typedef struct {
    int addr;
    uint32_t val;
} DataRec;
```

```
SymEntry symTab[MAX_SYM];
int symCount = 0;
```

```
InstRec instArr[MAX_INST];
int instCount = 0;
```

```
DataRec dataArr[MAX_DWORDS];
int dataCount = 0;
```

Parsing and Input Processing

- processInput - Orchestrates the reading and parsing of the assembly file. It differentiates between the text and data segments, setting the current address pointer accordingly. Depending on the segment, the function delegates line processing to either processTextLine or processDataLine.

```
// First pass: read file and build symbol table and text/data segment arrays
void processInput(FILE *fp) {
    char line[MAX_LINE];
    SegType seg = SEG_NONE;
    int curAddr = 0;
    while(fgets(line, MAX_LINE, fp)) {
        trimStr(line);
        if(strlen(line)==0) continue;
        if(strcmp(line, ".text") == 0) {
            seg = SEG_TEXT;
            curAddr = MEM_TEXT_BEGIN;
            continue;
        } else if(strcmp(line, ".data") == 0) {
            seg = SEG_DATA;
            curAddr = MEM_DATA_BEGIN;
            continue;
        }
        if(seg == SEG_TEXT) {
            processTextLine(line, &curAddr);
        } else if(seg == SEG_DATA) {
            processDataLine(line, &curAddr);
        }
    }
}
```

- processTextLine - Processes the instructions and/or labels in the text segment of the assembly file. It looks for a colon in each line to identify any labels and adds them to the symbol table, before processing the remaining instruction.

```
// Process a line from the text segment
void processTextLine(char *line, int *curAddr) {
    char *colon = strchr(line, ':');
    if(colon) {
        char tmpLbl[MAX_LBL];
        int len = colon - line;
        strncpy(tmpLbl, line, len);
        tmpLbl[len] = '\0';
        trimStr(tmpLbl);
        strcpy(symTab[symCount].lbl, tmpLbl);
        symTab[symCount].addr = *curAddr;
        symCount++;
        char *rest = colon+1;
        trimStr(rest);
        if(strlen(rest) > 0) {
            strcpy(instArr[instCount].instLine, rest);
            instArr[instCount].addr = *curAddr;
            instCount++;
            *curAddr += 4;
        }
    } else {
        strcpy(instArr[instCount].instLine, line);
        instArr[instCount].addr = *curAddr;
        instCount++;
        *curAddr += 4;
    }
}
```

- processDataLine - Handles the parsing of data segment lines and tokenizes values following the .word directive.

```
// Process a line from the data segment
void processDataLine(char *line, int *curAddr) {
    char *colon = strchr(line, ':');
    if(colon) {
        char tmpLbl[MAX_LBL];
        int len = colon - line;
        strncpy(tmpLbl, line, len);
        tmpLbl[len] = '\0';
        trimStr(tmpLbl);
        strcpy(symTab[symCount].lbl, tmpLbl);
        symTab[symCount].addr = *curAddr;
        symCount++;
        char *rest = colon+1;
        trimStr(rest);
        if(strncmp(rest, ".word", 5) == 0) {
            rest += 5;
            char *tok = strtok(rest, ",");
            while(tok) {
                trimStr(tok);
                dataArr[dataCount].addr = *curAddr;
                dataArr[dataCount].val = strtol(tok, NULL, 0);
                dataCount++;
                *curAddr += 4;
                tok = strtok(NULL, ",");
            }
        }
    } else if(strncmp(line, ".word", 5) == 0) {
        char *rest = line+5;
        char *tok = strtok(rest, ",");
        while(tok) {
            trimStr(tok);
            dataArr[dataCount].addr = *curAddr;
            dataArr[dataCount].val = strtol(tok, NULL, 0);
            dataCount++;
            *curAddr += 4;
            tok = strtok(NULL, ",");
        }
    } else {
        printf("Data error: expected .word\n");
        exit(1);
    }
}
```

Instruction Encoding

The program defines separate functions for each RISC-V instruction format. They shift and combine the variables for *funct7*, *funct3*, *rs1*, *rs2*, *rd*, and the *opcode* into one 32-bit hexadecimal value according to the specifications of each format.

```
uint32_t encode_R(uint8_t opcode, uint8_t rd, uint8_t funct3, uint8_t rs1, uint8_t rs2, uint8_t funct7) {
    uint32_t instruction = 0;

    // Shift and place each field into its position
    uint32_t funct7_part = ((uint32_t)funct7) << 25;
    uint32_t rs2_part = ((uint32_t)rs2) << 20;
    uint32_t rs1_part = ((uint32_t)rs1) << 15;
    uint32_t funct3_part = ((uint32_t)funct3) << 12;
    uint32_t rd_part = ((uint32_t)rd) << 7;

    // Construct the final instruction
    instruction |= funct7_part;
    instruction |= rs2_part;
    instruction |= rs1_part;
    instruction |= funct3_part;
    instruction |= rd_part;
    instruction |= opcode;

    //printf("Encoding R-Type: funct7=%02X, rs2=%02X, rs1=%02X, funct3=%02X, rd=%02X, opcode=%02X -> %08X\n",
    return instruction;
}
```

```
uint32_t encode_I(uint8_t opcode, uint8_t rd, uint8_t funct3, uint8_t rs1, int32_t imm) {
    uint32_t instruction = 0;
    uint32_t imm_part = (imm & 0xFFFF) << 20;
    uint32_t rs1_part = ((uint32_t)rs1) << 15;
    uint32_t funct3_part = ((uint32_t)funct3) << 12;
    uint32_t rd_part = ((uint32_t)rd) << 7;

    instruction |= imm_part;
    instruction |= rs1_part;
    instruction |= funct3_part;
    instruction |= rd_part;
    instruction |= opcode;

    //printf("Encoding I-Type: imm=%X, rs1=%02X, funct3=%02X, rd=%02X, opcode=%02X -> %08X\n", imm, r
    return instruction;
}
```

```

uint32_t encode_S(uint8_t opcode, uint8_t funct3, uint8_t rs1, uint8_t rs2, int32_t imm) {
    uint32_t lowBits = imm & 0x1F;
    uint32_t highBits = (imm >> 5) & 0x7F;
    return (highBits << 25) | (((uint32_t)rs2) << 20) | (((uint32_t)rs1) << 15) | (((uint32_t)funct3) << 12) | (lowBits << 7) | opcode;
}

uint32_t encode_B(uint8_t opcode, uint8_t funct3, uint8_t rs1, uint8_t rs2, int32_t offset) {
    if (offset % 2 != 0) {
        fprintf(stderr, "Error: Branch offset must be even. Given: %d\n", offset);
        return 0;
    }

    uint32_t instruction = 0;
    uint32_t bit12 = (offset >> 12) & 0x1;
    uint32_t bit11 = (offset >> 11) & 0x1;
    uint32_t bits10_5 = (offset >> 5) & 0x3F;
    uint32_t bits4_1 = (offset >> 1) & 0xF;

    instruction |= (bit12 << 31);
    instruction |= (bits10_5 << 25);
    instruction |= ((uint32_t)rs2 << 20);
    instruction |= ((uint32_t)rs1 << 15);
    instruction |= ((uint32_t)funct3 << 12);
    instruction |= (bits4_1 << 8);
    instruction |= (bit11 << 7);
    instruction |= opcode;

    //printf("Encoding B-Type: offset=%X, rs1=%02X, rs2=%02X, funct3=%02X, opcode=%02X -> %08X\n", offset, rs1, rs2, funct3, opcode, inst

    return instruction;
}

uint32_t encode_U(uint8_t opcode, uint8_t rd, int32_t imm) {
    return (((imm & 0xFFFFF) << 12) | (((uint32_t)rd) << 7) | opcode;
}

uint32_t encode_J(uint8_t opcode, uint8_t rd, int32_t offset) {
    if (offset % 2 != 0) {
        printf("Jump offset error\n");
        exit(1);
    }
    uint32_t bit20 = (offset >> 20) & 0x1;
    uint32_t bits10_1 = (offset >> 1) & 0x3FF;
    uint32_t bit11 = (offset >> 11) & 0x1;
    uint32_t bits19_12 = (offset >> 12) & 0xFF;
    return (bit20 << 31) | (bits19_12 << 12) | (bit11 << 20) | (bits10_1 << 21) | (((uint32_t)rd) << 7) | opcode;
}

```